

ECC2K-130 on Blackwell GPUs

Native Carryless Multiplication, Polynomial State,
and Cooperative Batch Inversion

Adam Buran

October 1, 2026

Research preprint draft, revision 2

Abstract

In 2009 Bailey et al. estimated that solving the ECC2K-130 challenge, a 130-bit discrete logarithm on a Koblitz curve over $\mathbb{F}_{2^{131}}$, takes about $2^{60.9}$ iterations of Pollard’s rho method, and a 2010 implementation reached 63 million iterations per second on a GTX 295 graphics card. We revisit the problem on an NVIDIA RTX PRO 6000 Blackwell GPU, whose `clmad` instruction provides native 64×64 -bit carryless multiplication. Field elements are held in a polynomial basis generated by the same element as the type-II optimal normal basis, so products use three-way Karatsuba over `clmad` while the normal-basis weight that drives the walk stays cheap to evaluate. Inversions are shared through weighted prefixes within a thread and a product tree across four warps, for $5 + 5/(BW)$ field multiplications per update.

Running the exact iteration function of Bailey et al., with distinguished-point collection active, the implementation reaches a median of **17.625650 billion iterations per second** over five paired runs, with every run producing the same 705-record corpus and no drops. That is about 280 times the published GTX 295 rate per card and about 25 SM clock cycles per iteration. Bailey et al.’s $2^{60.9}$ estimate for this walk is supported by measured iteration constants on scaled curves, so the expected work is about 3.9 GPU-years on one card. We also report an eight-branch table walk that removes the per-step Frobenius work and reaches **22.100934 billion scheduled updates per second**, 11.3% over its same-session control, using a compressed even/odd square-and-reduce circuit that we prove exact. We show, however, that the cycle-avoidance rule in that measured build admits four-step cycles arising from the Frobenius relation $\sigma^2 + \sigma + 2 = 0$. These trap about 12% of trails at the benchmark’s distinguished-point threshold, so the 22.1 figure is an arithmetic ceiling, not a faster solver. We do not claim a solution of ECC2K-130 or a new asymptotic algorithm.

Keywords: ECC2K-130; Koblitz curves; GPU cryptanalysis; binary fields; batch inversion; carryless multiplication; reproducible benchmarking.

1 Introduction

The ECC2K-130 challenge provides a concrete setting in which to study the cost of elliptic-curve computation over a binary field. Its arithmetic is especially suitable for low-level optimization: field addition is XOR, the Frobenius endomorphism is linear, and many independent walks can share inversions. These features also create a difficult implementation tradeoff. A larger batch saves inversions but increases the working set; more parallel work can hide latency but consumes registers and shared memory; a cheaper iteration function can change the statistical behavior on which collision search depends.

GPUs have lacked a carryless multiplier, which is why the 2010 GPU implementation bitsliced its arithmetic [2]. NVIDIA’s PTX ISA 9.3 adds `c1mad`, a 64×64 -bit carryless multiply-add, on every target from `sm_80` [20, 19]. This paper asks what that instruction, together with the memory and scheduling design it makes possible, does for the cost of ECC2K-130. The measurements are complete walk loops on one GPU, including selection, distinguished-point handling and coordinate updates, rather than isolated multiplication benchmarks. The source and measurement receipts are fixed by content hashes.

The contributions are:

1. A packed representation of $\mathbb{F}_{2^{131}}$ for GPUs: polynomial state in the basis generated by the type-II optimal normal-basis element, so that `c1mad` Karatsuba products and the normal-basis weight used by the walk are both cheap.
2. A two-level inversion schedule whose multiplication budget is $5 + 5/(BW)$ per ordinary affine update, with $B = 16$ slots per thread and $W = 4$ collaborating warps, and an exact even/odd-stream square-and-reduce map for the degree-131 internal modulus, with a basis-vector proof.
3. A paired, five-run measurement of the campaign walk, Bailey et al.’s Frobenius iteration, at 17.625650 billion iterations/s with collection active, and the resulting cost to solve ECC2K-130: about 3.9 GPU-years on one card.
4. A paired, five-run measurement of an eight-branch table walk at 22.100934 billion scheduled updates/s, with a screening ladder and resource analysis.
5. An analysis of why that table walk is not yet a faster solver: its measured cycle rule admits fruitless cycles from the Frobenius relation $\sigma^2 + \sigma + 2 = 0$, which trap about 12% of trails at the benchmark’s threshold. We give the counts, the cost per solve, and the status of the repaired rule.

We use *state of the art* only as literature context. A global fastest-solver claim would require a current, comparable survey across hardware, which we do not supply.

2 Background and comparison scope

Pollard’s rho method provides the classical low-memory approach to generic discrete logarithms [21]. Distinguished-point methods permit many processors to contribute trails to a common collision search [22]. In that setting, iteration speed is only one component of performance: trail coalescence, fruitless cycles, endpoint production, replay, and collision usefulness all affect the work needed for a successful computation.

On Koblitz curves the Frobenius map lets a walk operate on classes $\{\pm\sigma^k(R)\}$ of size $2m$, which reduces the expected work by $\sqrt{2m}$ [25, 16]. The negation part of that reduction creates fruitless cycles, analyzed by Bos, Kleinjung and Lenstra [7] and by Bernstein, Lange and Schwabe [6]. The ECC2K-130 work of Bailey et al. chose an iteration function, $R \mapsto R + \sigma^j(R)$, that is well defined on classes and has no fruitless cycles, and estimated $2^{60.9}$ iterations [1]. Implementations followed on Cell processors (25.57 million iterations/s on the six usable SPUs of a PlayStation 3 [8]), on GPUs (more than 63 million iterations/s on a GTX 295, bitsliced, after hand-written assembly [2, 3]), and on FPGAs [15]. Later FPGA work solved discrete logarithms on 113-bit binary curves [23, 24] and a 117.35-bit discrete logarithm on a binary-field curve [4]. To our knowledge ECC2K-130 itself has not been solved.

Batch inversion and normal-basis exponentiation are established techniques [18, 17], as are type-II optimal polynomial bases [5]. Our contribution is a concrete implementation built on a new instruction, its measured combination of optimizations, and an analysis of what its faster walk would cost per solve. We do not claim to introduce Pollard rho, Frobenius classes, table walks in general, or the batch-inversion principle.

Work	Hardware / walk	Rate	Unit
Bos et al. [8]	Cell, six SPUs; Bailey et al. walk	25.57	million/s
Bernstein et al. [2]	GTX 295 card; Bailey et al. walk	> 63	million/s
Bitsliced control	RTX PRO 6000; Bailey et al. walk	0.852294	billion/s
This work	RTX PRO 6000; Bailey et al. walk, collecting DP32	17.625650	billion/s
Table walk, control	RTX PRO 6000; eight-branch table walk	19.852741	billion/s
Table walk, this work	Same; arithmetic ceiling (Section 3.5)	22.100934	billion/s

Table 1: Contextual throughput comparison. The first two rows are published historical rates. The bitsliced control is a 2010-style kernel measured on the same GPU model with an earlier toolchain; it is not paired with any later row. Each of the two “this work” rows is the median of five runs paired with its own control in the same session (Section 7).

3 Curve, representations, and walk semantics

3.1 Curve and field

The implemented curve is

$$E : y^2 + xy = x^3 + 1 \quad \text{over } \mathbb{F}_{2^{131}}, \quad (1)$$

with the prime-order subgroup parameter recorded in the generated source as

$$\ell = 680564733841876926932320129493409985129. \quad (2)$$

For an affine point $R = (x, y)$, negation is $-R = (x, y + x)$ and Frobenius is $\sigma(R) = (x^2, y^2)$. The ordinary automorphism orbit is $\{\pm\sigma^k(R) : 0 \leq k < 131\}$, of size at most 262. Orbit arithmetic and exceptional orbit sizes must be distinguished from a heuristic uniform random map on $\ell/262$ objects.

The code uses a permuted type-II optimal normal basis indexed by $e \in \{1, \dots, 131\}$, with $263 = 2 \cdot 131 + 1$ and the index operation

$$\text{fold}(u) = \min(u \bmod 263, 263 - (u \bmod 263)). \quad (3)$$

Squaring maps index e to $\text{fold}(2e)$. A normal-basis coordinate vector is useful for Hamming weight and walk selection. Persistent coordinates and most multiplication intermediates instead use an internal polynomial basis; conversion is an exact generated linear map.

The internal reduction polynomial is represented by the bit string

$$F = 0xD1D0D000D0000000D00000000000000D. \quad (4)$$

Equivalently, with the bit in position i denoting the coefficient of z^i ,

$$F(z) = z^{124} + (1 + z^2 + z^3)(1 + z^{64} + z^{96} + z^{112} + z^{120} + z^{128}). \quad (5)$$

This internal representation is distinct from the sparse polynomial $z^{131} + z^{13} + z^2 + z + 1$ also listed by the parameter generator, and the choice is structural. Let ζ be a primitive 263rd root of unity and $\beta = \zeta + \zeta^{-1}$, so that the normal-basis elements are $\beta_e = \zeta^e + \zeta^{-e}$. The minimal polynomial of β over $\mathbb{F}_2$ is produced by the recurrence

$$f_0 = 1, \quad f_1 = z + 1, \quad f_{j+1} = zf_j + f_{j-1}, \quad (6)$$

and a direct computation confirms $f_{131} = F$ exactly. The internal polynomial basis is therefore $1, \beta, \dots, \beta^{130}$: the same element β generates both representations. Since

$$\beta^j = \sum_{0 \leq i < j/2} \binom{j}{i} \beta_{j-2i} \quad \text{in characteristic two,} \quad (7)$$

the change of basis is a fixed and highly structured linear map, closely related to the type-II optimal polynomial bases of Bernstein and Lange [5]. This is what makes it affordable to keep persistent state in polynomial form while still evaluating the normal-basis weight and selector on every step.

The price is a denser modulus: F has 19 terms where the pentanomial has five. The factored form above limits that cost, because reduction applies only the three-term factor $1+z^2+z^3$ at six byte-aligned offsets, together with the single term z^{124} . With the pentanomial, the selector would instead need an unstructured 131×131 basis change on every step. That alternative was not measured, so the tradeoff is offered as the design rationale and not as a measured optimum. The square circuit below is tied to Equation (4); substituting another representation without changing the linear maps would be incorrect.

3.2 The Frobenius walk

The campaign walk is the iteration function of Bailey et al. [1]:

$$g(R) = \sigma^{j(R)}(R) + R, \quad j(R) = 3 + \left(\frac{\text{HW}(x_{\text{NB}})}{2} \bmod 8 \right). \quad (8)$$

The normal-basis weight is invariant under σ and under negation, so g is well defined on the classes $\{\pm\sigma^k(R)\}$: $g(\pm\sigma^k R) = \pm\sigma^k g(R)$. A walk is identified by the canonical representative of its class at each distinguished point. Because the step adds a multiple $(1 + \tau^j)$ of the current point, a cycle would need $\prod_i (1 + \tau^{j_i}) = \pm\tau^a$ in the endomorphism ring, which has no formal solutions; the walk has no fruitless cycles [12]. Its iteration constant was measured on the same branch distribution over scaled curves of degree $n = 23$ to 59 at $c = 1.08$ – 1.10 times a random mapping, falling towards its first-order value $1/\sqrt{1 - \sum_j p_j^2} = 1.070$ [12]. The expected work at $n = 131$ is therefore $2^{60.81} \cdot [1.070, 1.082] = 2^{60.91}$ – $2^{60.92}$ iterations, in line with Bailey et al.’s $2^{60.9}$.

Each step evaluates σ^j on both coordinates. The implementation applies the Frobenius powers as generated bit-permutation networks whose masks are held in shared memory, and computes $\sigma^j(R) + R$ with the affine formulas and shared inversion of Section 4. This per-step Frobenius and basis work is what the table walk below removes.

3.3 Eight-branch table selection

Let P, Q be the public challenge points. The reference implementation deterministically constructs $H = 8$ table points $T_h = [a_h]P + [b_h]Q$, along with all $\sigma^k(T_h)$. The coefficients are fixed by a public branch-index seed and modular reduction; they are part of the walk identity. The device reads the table coordinates in polynomial representation.

Write the normal-basis x coordinate as (x_e) . Define L by

$$L(\text{fold}(2^t)) = t, \quad 0 \leq t < 131. \quad (9)$$

For $w = \sum_e x_e$ with $w \not\equiv 0 \pmod{131}$, the raw selection is

$$h(R) = \lfloor w/2 \rfloor \bmod 8, \quad (10)$$

$$k(R) = w^{-1} \sum_e L(e) x_e \pmod{131}, \quad (11)$$

$$p(R) = \arg \max_{e: x_e=1} ((L(e) - k(R)) \bmod 131), \quad (12)$$

$$\epsilon(R) = y_{p(R)}. \quad (13)$$

Before the cycle correction, the update is

$$f(R) = R + (-1)^{\epsilon(R)} \sigma^{k(R)}(T_{h(R)}). \quad (14)$$

Only the selected normal-basis coordinate of y is needed: a precomputed linear row evaluates it from polynomial y , avoiding a full conversion. The x conversion is retained for the selector and distinguished-point test.

Proposition 1 (Raw-selector equivariance). *On the domain where the selector and affine update are defined, the raw map in Equation (14) satisfies $f(\sigma R) = \sigma f(R)$ and $f(-R) = -f(R)$.*

Proof. Squaring permutes the support and adds one to every $L(e)$ modulo 131. The weight is unchanged, so k increases by one and the cyclic ordering defining the pivot is preserved. The selected y bit is carried along with its pivot, making ϵ Frobenius-invariant. Under negation x does not change, while y becomes $y + x$. The pivot lies in the support of x , hence its y bit flips. Applying these transformations to the chosen addend gives the two identities. $\square$

The statement has an explicit domain: a zero coordinate has no pivot, and weights divisible by 131 do not admit the displayed inverse. Inside the prime-order subgroup this part of the domain is never left. Every subgroup point is a double, so $\text{Tr}(x) = \text{Tr}(a_2) = 0$ for this curve, whose x^2 coefficient a_2 is zero. Each element of the normal basis has trace one, hence $\text{Tr}(x) \equiv w \pmod{2}$ and the weight w is always even. This is also why the branch index uses $\lfloor w/2 \rfloor$. Since $0 \leq w \leq 131$ and 131 is odd, $w \equiv 0 \pmod{131}$ forces $w = 0$, that is $x = 0$. The only affine point with $x = 0$ is $(0, 1)$, which has order two and does not lie in the subgroup of order ℓ . The selector is therefore defined at every point a trail can reach. The remaining exceptional case is the affine addition itself, which is treated in Section 8.2.

3.4 History-dependent cycle avoidance

The measured table-walk build augments the point with the three most recent step tags. A tag is (h, k, ϵ) , packed into a 16-bit value. If the proposed tag is the sign-opposite of the preceding tag, it would immediately undo that addition. The code also rejects the four-step pattern where the new tag cancels the second previous tag and the most recent tag cancels the third previous tag. It increments h modulo eight until the test clears, then updates history. At most two branch values can be prohibited by these two equality tests for a fixed (k, ϵ) , so the branch scan terminates.

The resulting transition is a function of *point and history*. Two trails at the same point can select different successors when their histories differ. Equivariance extends to consistently transformed histories, but it does not by itself prove that projected point collisions coalesce as in a point-only rho walk.

3.5 Fruitless cycles the rule admits

The table walk is additive, so any run of steps whose addends sum to zero returns a trail to a point it has already visited. The rule above refuses only pairwise cancellations. It misses a family that is specific to Koblitz curves. On this curve the Frobenius endomorphism satisfies

$$\sigma^2 + \sigma + 2 = 0, \quad \text{hence} \quad \sigma^3 + \sigma - 2 = 0. \quad (15)$$

Four steps in one branch h whose addends are $\sigma^{k+2}T_h, \sigma^{k+1}T_h, \sigma^kT_h, \sigma^kT_h$ with one sign, or $\sigma^{k+3}T_h, \sigma^{k+1}T_h, -\sigma^kT_h, -\sigma^kT_h$, therefore sum to the identity in any order. No two of these tags cancel, so the rule never fires. An enumeration of the patterns the rule admits finds 24 such four-step τ -relation cycles and four six-step pairwise cycles such as $a, b, c, -a, -b, -c$, both at leading order $(2 \cdot 131)^{-3}$, and nothing at two, three or five steps [12]. A trapped trail repeats its cycle indefinitely, because its tag history at the start of each lap is the same, and never becomes distinguished.

Writing p_h for the branch probabilities, a trail enters a τ -relation cycle with probability about $24 \sum_h p_h^4 / (262)^3$ per step and a six-step pairwise cycle with probability about $4(\sum_h p_h^2 / 262)^3$. With eight equiprobable branches these are 2.6×10^{-9} and 4.3×10^{-10} . Measured on scaled curves, on the device walk and on an emulation, the τ -relation count matched the prediction: 1,145 returns observed against 1,312 predicted, and 65 against 70 on the device at $n = 23$ [12]. At $n = 131$ with ECC2K-130’s branch distribution, the resulting trap model gives:

Distinguished-point threshold (mean trail)	Trails trapped	Loss at best restart guard
DP34 ($2^{25.27}$ steps; this paper’s benchmark)	11.6%	$\times 1.655$
DP32 ($2^{28.41}$ steps; the live campaign)	53.7%	$\times 5.33$

A trapped trail runs to the restart guard (`-max-iters`) and emits nothing, so its steps are lost. An earlier version of this manuscript modelled only the six-step family and estimated about 1% trapped at DP34. That was about ten times too low.

Priced end to end at DP32, with the iteration constants and paired rates of both walks, the table walk as measured costs 4.85–6.26 times the Frobenius walk per solve [12]. A walk that refused the τ -relations as well would cost 1.36–1.52 times, because the six-step cycles remain. A walk that detects every short cycle and leaves it deterministically is projected at 0.81–0.86 times. A repaired rule of that kind now exists. It adds both τ -relation families and a four-tag window, and on a hint follows at most eight raw steps, confirms an exact return, and leaves the cycle at a canonical anchor chosen from the cycle itself, so that every trail in a cycle exits the same way [14]. Its GPU throughput has not been measured, and the 22.1 billion/s figure below does not transfer to it.

Merges also part slightly under a history rule. Under the intermediate four-tag rule, before the anchor was added, two walks that meet with different histories take different successors 0.97% of the time, which costs a factor of about 1.0048 in iterations [12].

3.6 Distinguished points and replay

The collection predicate is $\text{HW}(x_{\text{NB}}) \leq 34$. Thus “DP34” denotes a weight threshold; it does not denote a probability of 2^{-34} . Because the weight is always even, the probability that a uniform subgroup point is distinguished is

$$\theta_{34} = 2^{-130} \sum_{\substack{0 \leq i \leq 34 \\ i \text{ even}}} \binom{131}{i} \approx 2.472 \times 10^{-8} = 2^{-25.27}, \quad (16)$$

so a trail has mean length $1/\theta_{34} \approx 4.0 \times 10^7$ steps. The same threshold was used by Bailey et al. [1]. A device report includes its seed, trail length, and endpoint coordinates. The host replay starts from the same seed and checks both coordinates and the number of steps. The persisted report is 32 bytes: an eight-byte seed and a 24-byte canonical orbit key.

The benchmark sets the threshold to zero and disables reference replay in the timed loop. Correctness replay is performed separately with a higher reporting threshold. This separation avoids confusing the performance of host reference arithmetic with GPU update throughput.

4 Packed arithmetic and cooperative inversion

4.1 Carryless multiplication and affine updates

An element is held in five 32-bit words, with only three significant bits in the fifth word. A polynomial product has degree at most 260. The selected multiplier uses a 128-bit Karatsuba decomposition: three 64-by-64 carryless products, each supplied by low and high `clmad`

Algorithm 1 One scheduled batch step, ordinary nonzero-denominator domain

Require: Slots $(x_i, y_i, \mathcal{H}_i)$, $0 \leq i < B$; fixed table; W -warp group

Ensure: Updated coordinates and history; any endpoint reports buffered

```

1:  $D \leftarrow 1$ 
2: for  $i = 0, \dots, B - 1$  do
3:   Evaluate weight and endpoint predicate; update report/dead metadata
4:    $t_i \leftarrow \text{SELECTTAG}(x_i, y_i, \mathcal{H}_i)$ ; push  $t_i$  into  $\mathcal{H}_i$ 
5:    $(u_i, v_i) \leftarrow \text{TABLEADDEND}(t_i)$ ;  $d_i \leftarrow x_i + u_i$ ;  $e_i \leftarrow y_i + v_i$ 
6:    $W_i \leftarrow e_i D$ ;  $D \leftarrow D d_i$  ▷ Omit products by 1 at  $i = 0$ 
7: end for
8:  $v \leftarrow \text{GROUPINVERSE}(D, W)$  ▷ One independent tree per lane
9: for  $i = B - 1, \dots, 0$  do
10:  Reconstruct  $d_i$  from  $t_i$  and original  $x_i$ 
11:   $\lambda \leftarrow v W_i$ ; if  $i > 0$ , set  $v \leftarrow v d_i$ 
12:   $x'_i \leftarrow \text{SQUAREREDUCE}(\lambda) + \lambda + d_i$ 
13:   $y'_i \leftarrow \lambda(x_i + x'_i) + x'_i + y_i$ 
14:  Store  $(x'_i, y'_i)$ 
15: end for

```

instructions, followed by the corrections for the three high input bits and exact modular reduction. NVIDIA specifies `clmad.lo.u64` and `clmad.hi.u64` as carryless multiply-add operations; the PTX instruction was introduced in ISA 9.3 and is available for targets at least `sm_80` [20]. This paper’s measured build targets `sm_120`.

For a selected addend (u, v) , define $d = x + u$ and $e = y + v$. On the ordinary affine domain $d \neq 0$,

$$\lambda = e/d, \quad x' = \lambda^2 + \lambda + d, \quad y' = \lambda(x + x') + x' + y. \quad (17)$$

The implementation retains polynomial state throughout these formulas. In particular, it need not convert both coordinates to normal basis for every addition.

4.2 Weighted prefixes within a thread

Each thread owns $B = 16$ independent scalar slots. For slot i , let

$$D_i = \prod_{j=0}^i d_j, \quad W_i = e_i \prod_{j=0}^{i-1} d_j. \quad (18)$$

The forward pass stores W_i and retains D_{B-1} . Given $v = D_{B-1}^{-1}$, a reverse pass recovers $\lambda_i = v W_i$ and updates $v \leftarrow v d_i$ before proceeding to the previous slot. The denominator is reconstructed from the stored tag and the original x_i , avoiding a separate denominator field in global memory.

The forward pass uses $2(B - 1)$ multiplications. The reverse pass uses B slope products, $B - 1$ inverse-chain products, and B products for the y update. Consequently the ordinary update work, excluding the inverse, totals $5B - 3$ multiplications. The root inverse follows an Itoh–Tsujii chain using eight multiplications: the exponents represented by $\beta_j = a^{2^j - 1}$ progress through $j = 2, 4, 8, 16, 32, 64, 65, 130$, followed by a square. This yields $5B + 5$ multiplications per isolated thread batch.

4.3 Four-warp product trees

For each lane position, four warps contribute their final batch products to a shared-memory product tree. Combining W products and distributing the inverse costs $3(W - 1)$ multiplica-

tions and one root inversion. The arithmetic budget across W batches is therefore

$$W(5B - 3) + 3(W - 1) + 8 = 5BW + 5, \quad (19)$$

or

$$5 + \frac{5}{BW} \quad \text{field multiplications per update.} \quad (20)$$

For $B = 16$, changing $W = 1$ to $W = 4$ reduces this count from 5.3125 to 5.078125, a 4.41% reduction. This is a field-operation count, not a throughput prediction: it excludes barriers, data movement, squarings, table selection, and register effects.

The implementation replaces zero input *batch products* with one before forming the cross-warp tree, then returns zero for those original inputs. This prevents a zero in one thread's batch product from contaminating other threads' inverse results. It does not repair an exceptional affine addition inside the affected thread's batch.

The accepted helper uses a separate named barrier for each group of four complete warps. A 640-thread block contains five such groups. Within each group, logical warp indices are rotated by the group index; the physical roots become warps 3, 6, 9, 12, and 19. This permutation preserves the field results and redistributes the root work. A claimed mapping from those warp identifiers to undocumented physical scheduler assignments is unnecessary and is not assumed.

For $W = 4$, the helper uses seven group barriers per call, including a final barrier that protects scratch reuse. The scratch occupies $5 \cdot 640 \cdot 4 = 12,800$ bytes per block. Partial blocks bypass the collective path and use an individual inverse; the collective barriers are entered only by full groups.

4.4 Square reduction in compressed coefficient streams

In characteristic two, squaring has no cross terms. Define

$$\text{spread} \left(\sum_i a_i 2^i \right) = \sum_i a_i 2^{2i}. \quad (21)$$

The unreduced square of a polynomial bit vector a is $\text{spread}(a)$. A general product reducer accepts up to 261 bits, but a square has zero odd coefficients. The accepted circuit performs the reduction in compressed coefficient streams and interleaves only the final 131-bit answer.

All additions in the following expressions are XOR; shifts operate on nonnegative bit vectors. Set

$$b = a \gg 66, \quad (22)$$

$$E = b \oplus (b \gg 4) \oplus (b \gg 12) \oplus (b \gg 28) \oplus (b \gg 60), \quad (23)$$

$$O = b \oplus (E \gg 1) \oplus (E \gg 2), \quad (24)$$

$$T_E = E \oplus (E \ll 1) \oplus (O \ll 2), \quad (25)$$

$$T_O = O \oplus (O \ll 1) \oplus (E \ll 1). \quad (26)$$

With $S = \{0, 32, 48, 56, 60, 64\}$, compute

$$V_E = \left[a \oplus (E \ll 62) \oplus \bigoplus_{s \in S} (T_E \ll s) \right]_{66}, \quad (27)$$

$$V_O = \left[(O \ll 62) \oplus \bigoplus_{s \in S} (T_O \ll s) \right]_{65}, \quad (28)$$

$$\text{SquareReduce}(a) = \text{spread}(V_E) \oplus (\text{spread}(V_O) \ll 1), \quad (29)$$

where $[\cdot]_r$ truncates to the low r bits. The generated C++ implements these expressions in 32-bit words. Its final zip uses masked exchanges and byte permutation to interleave the even and odd streams; it does not materialize the complete unreduced square.

Proposition 2 (Exactness of the generated square map). *The bit-vector map in Equation (29) equals reduction of spread(a) modulo Equation (4) for every 131-bit input a .*

Computer-assisted linear-map proof. Both maps are linear over $\mathbb{F}_2$: they consist of coefficient spreading, fixed shifts, truncation, XOR, and reduction by a fixed polynomial. It suffices to compare them on the 131 unit vectors. The supplied generator performs that complete basis comparison against independent polynomial long division. Its check additionally covers zero, all ones, and 10,000 deterministic random values. The manuscript’s local verification reran these checks and regenerated the exact archived header byte for byte. The basis comparison proves the algebraic identity of the two bit-vector maps; the generated C++ and GPU execution are supported separately by implementation probes. $\square$

The local analysis also checks that the internal modulus equals the polynomial reconstructed by the general reducer and passes the binary irreducibility criterion for prime degree 131. These local checks are new analyses of the archived artifact; they are not new GPU timing measurements.

5 GPU storage and scheduling

The accepted persistent field layout stores 256 aligned 16-byte low-part records followed by 256 one-byte tails. Each field tile occupies 4,352 bytes, equivalent to 17 bytes per field element. Coordinates and weighted prefixes use three such field arrays, for 51 bytes per scalar slot before metadata. The baseline’s 1,540,096 slots therefore require 78,544,896 field bytes; the candidate’s 1,925,120 slots require 98,181,120 bytes.

Live metadata uses one byte per dead flag and three coalesced planes of 16-bit history tags. The unused fourth history tag is canonicalized when needed. The accepted byte-flag configuration is distinct from earlier experimental bit-packed flags. Checkpoint transfer hooks retain the archived external representation, which is why cross-build continuation is a useful compatibility test.

The kernel receives disjoint coordinate, prefix, metadata, constant, and report buffers through explicit restricted pointers. The underlying allocations justify the non-aliasing contract. The purpose is to permit the compiler to retain and move loads across stores when safe; the annotation is not an arithmetic optimization in itself. The accepted configuration disables additional forward selection pipelining and paired-product instruction interleaving, while retaining paired products and inlined polynomial helpers. The empirical choice reflects the balance between available work and live register pressure.

The receipt reports 94 registers per candidate thread and zero local bytes, compared with 116 registers and zero local bytes for the baseline. Both use 48,732 bytes of dynamic table storage. The candidate adds 12,800 bytes of static inverse scratch, with 1,024 bytes of driver-reserved shared storage. Its total is thus 62,556 bytes per block including the reservation. One block is resident per SM: 20 warps for the candidate and 16 for the baseline. These resource observations explain feasible occupancy; they do not alone establish the mechanism responsible for the measured gain.

The measured device has 188 SMs, 128 MiB of L2, an 80 MiB persisting-L2 allowance, 100 KiB of shared memory per SM, and 65,536 registers per SM. The candidate’s field window exceeds the persisting allowance; the reported requested persistence covers 83,886,080 of its 98,181,120 field bytes. No assumption that all state remains in a particular cache is needed for the reported timing.

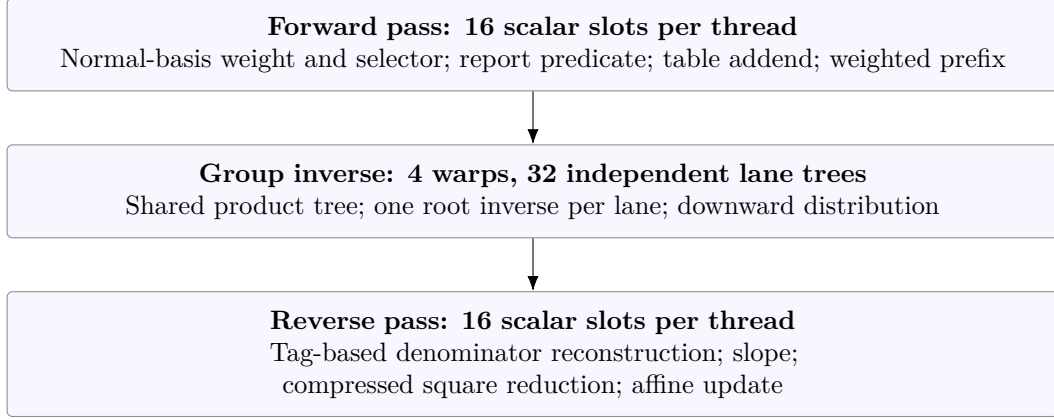


Figure 1: One step of the accepted two-pass kernel. Five four-warp groups fit in each 640-thread block. A lane tree shares an arithmetic inverse; it does not add independent walk points together.

6 Experimental methodology

6.1 Artifact and environment

There are two receipts, both from September 21, 2026, on the same GPU model and toolchain. The Frobenius-walk confirmation receipt records 26 build settings, the source hashes of the candidate, the binary hashes of control and candidate, every command and its full output, and 17 checkpoint and corpus compatibility checks [11]. The table-walk acceptance receipt and exported source tree [10] record the same kinds of evidence plus independent arithmetic probes. The GPU is an NVIDIA RTX PRO 6000 Blackwell Server Edition, driver 580.95.05, with a configured 600 W power limit. The compiler identifies itself as CUDA 13.3, v13.3.73. The archive contains compiler output, device identity, binary hashes, commands, full process output, probes, and the individual timing samples.

For the table walk, the manuscript verification recomputed the measurement hash and checked all 119 entries of the archived source manifest. Every entry matched. An imported repository HEAD is not used as a substitute for those source hashes: the source was imported from a working tree, and the current live configuration can change independently of the archived candidate.

6.2 Meaning of an update and the timing boundary

Both walks are measured by the same client and the same counter. The program’s completed-run count is

$$N = (I_{\text{end}} - I_{\text{start}}) T B, \quad (30)$$

where T is the number of worker threads and B the number of scalar slots per worker. The packed backend has one scalar point per slot; there is no extra factor of 32 from bitslicing. The Frobenius-walk runs use 120,320 workers, 1,024 steps per launch and 128 launches, so every run performs $120,320 \cdot 16 \cdot 1,024 \cdot 128 = 252,329,328,640$ iterations. The table-walk runs use 256 launches,

$$N_{\text{base}} = 96,256 \cdot 16 \cdot 1,024 \cdot 256 = 403,726,925,824, \quad (31)$$

$$N_{\text{cand}} = 120,320 \cdot 16 \cdot 1,024 \cdot 256 = 504,658,657,280. \quad (32)$$

We call these *scheduled scalar-slot updates*. Every counted slot runs the complete update arithmetic, including selection and coordinate stores. However, the kernel continues to com-

pute a slot after marking its trail finished until the launch ends and reseeding occurs. Such work is included in Equation (30). Its size is small at production settings: a slot finishes with probability θ_{34} per step (Equation (16)) and then idles for half a launch on average, so with 1,024 steps per launch the expected fraction of such updates is about $\theta_{34} \cdot 512 \approx 1.3 \times 10^{-5}$. In benchmark mode the threshold is zero and no slot finishes. Nevertheless a scheduled update must not be interpreted as a newly useful trail point, a distinct visited state, a field multiplication, or a complete scalar multiplication $[k]P$.

The host timer starts after initialization and any checkpoint restore. It covers the launch loop, report fetching and processing, and any reseeding, and ends after a final device synchronization. Thus these are completed client-loop rates, not CUDA-event timings of an isolated kernel. Collection timing includes host report handling and file writes inside that loop. The final file close is after the printed timing boundary. Startup, table construction, initial state generation, compilation, cloud uploads, database ingestion, and an entire collision-resolution campaign are not measured by this metric.

6.3 Confirmation protocol

Frobenius walk. The control is the deployed campaign binary; the candidate adds fused Frobenius passes, which prepare the next step’s denominator and weighted prefix while the updated point is still in registers, and issues the inverse-chain product before the slope product. After compilation the control runs 512 launches to warm the GPU. Five control/candidate pairs follow, with order alternating between pairs. Every run collects distinguished points at DP32, the campaign threshold, with host verification off. A valid run must reproduce the control’s corpus: all ten runs emitted the same 705 records, with one sorted SHA-256, and zero drops.

Table walk. Each variant receives an excluded full-workload warmup. The harness runs five confirmation rounds and reverses the order of baseline and candidate in alternating rounds. Both variants use the same GPU allocation, 1,024 steps per launch, 256 launches, and their respective normal worker geometry. A valid sample must exit successfully, supply a final completed-run rate, match the expected update count, and report zero dropped endpoints. The paper reports the median and range of the five samples.

These timings compare complete configurations, not individual instructions. Because the candidate has 25% more slots, the samples have different total update counts and durations. Dividing each count by its own measured duration makes a throughput comparison meaningful, while limiting attribution of the gain to any one component. Within the session the ordering is unambiguous: all five candidate samples exceed all five baseline samples, which under the null hypothesis of exchangeable samples has exact one-sided probability $1/\binom{10}{5} = 1/252 \approx 0.004$ (Wilcoxon rank-sum). That statement concerns this session only. Five repetitions from one device session are insufficient to estimate variation across machines, drivers, days, or production workloads, and we do not report a population confidence interval.

Higher short-screening results were deliberately excluded from the headline. In particular, the 22.610829- and 22.633932-billion-update/s three-sample screens in the research notebook were used to select a candidate; the new five-sample confirmation determines the reported result. The simpler plain square-reduction candidate was selected over the combined variant, whose small screening advantage was not established independently.

Pair	Deployed control, billion/s	Candidate, billion/s	Gain
1	17.193422	17.661360	2.7216%
2	17.158062	17.647467	2.8523%
3	17.156838	17.625650	2.7325%
4	17.136178	17.617125	2.8066%
5	17.138633	17.613956	2.7734%
Median	17.156838	17.625650	2.7734%

Table 2: Frobenius walk, Bailey et al.’s iteration function, collecting at DP32. Each run performs 252,329,328,640 iterations and emits the same 705 records; the gain column is each pair’s own ratio, and its median is the median of the five ratios.

Step	From	To	Gain
Bitsliced control	—	0.852294	—
Single-product multiplier, equal occupancy	2.956981	3.366572	+13.85%
Denominator cache, equal occupancy	3.506509	4.024103	+14.76%
Frobenius permutation networks	4.115575	5.811421	+41.21%
Polynomial-basis batch chain	5.832342	6.054107	+3.80%
Paired products	6.123614	6.215430	+1.50%
Native <code>clmad</code> , same layout*	7.110440	8.703518	+22.4%
Batch 16, compact state, shared Frobenius masks*	8.703518	14.637530	+68%
Deployed: four-warp inversion, 640-thread blocks [†]	—	17.156838	—
Fused Frobenius passes (Table 2)	17.156838	17.625650	+2.77%

Table 3: Packed Frobenius-walk steps, billion iterations/s on one RTX PRO 6000. Unmarked gains are each step’s own paired comparison; rows are not chained, and adjacent rows come from different sessions. The polynomial-chain and paired-product steps also change occupancy.

*Ratios of rates from separate receipts, used with their instruction counts in [9]; not paired.

[†]The deployed control of Table 2; no paired comparison against the 14.64 row exists.

7 Results

7.1 Frobenius walk: five paired runs

Table 2 gives every timed run. All five pairs improve, by 2.72–2.85%; the candidate median is **17.625650 billion iterations/s**. Every run lasts about 14.3 seconds. The candidate keeps 96 registers per thread, zero local bytes and 14,592 bytes of shared memory per 640-thread block. The receipt carries no clock telemetry. At the 2,362 MHz recorded in the table-walk session on the same GPU model, 17.63 billion/s is about 25 SM clock cycles per iteration; the 2010 GTX 295 implementation needed about 1,180 at its 60 SMs and 1.242 GHz [2].

This is the rate a campaign obtains: it is measured with collection on, on the walk whose iteration constant and absence of fruitless cycles are established (Section 3.2). The candidate’s build profile was deployed to the campaign’s workers on September 25, 2026, against this confirmation receipt.

7.2 From bitslicing to native carryless products

The packed backend was developed against a bitsliced control running the same iteration function on the same GPU model, which measured 0.852294 billion/s with an earlier toolchain [13]. Table 3 lists the steps that were each measured against an adjacent control.

Round	Baseline, billion/s	Candidate, billion/s
1	20.001884	22.166455
2	19.867106	22.132776
3	19.852741	22.100934
4	19.841065	22.093414
5	19.846254	22.089525
Median	19.852741	22.100934
Minimum	19.841065	22.089525
Maximum	20.001884	22.166455

Table 4: All accepted timing samples in recorded round order; one excluded warmup per variant. Every baseline sample performs 403,726,925,824 scheduled updates and every candidate sample performs 504,658,657,280.

The factor of about 21 from the bitsliced control to 17.63 billion/s belongs to the whole packed design, not to `clmad` alone. Instruction counts make the division of labor visible [9]. The software carryless product was already issue-bound, at 69.1 lane-instructions per SM clock against 69.3 measured for the matching instruction mix. Replacing it with `clmad` cut instructions per update by 47% but bought only 22.4% of throughput, because the kernel then stalled on state traffic and dropped to 44.9 lane-instructions per clock. The layout work that followed changed the instruction count by less than 2% and raised throughput by 68%, returning the kernel to 74.1 lane-instructions per clock. `clmad` created the headroom; batching and layout consumed it.

7.3 Table walk: five-sample confirmation

The table-walk rows that follow measure the arithmetic throughput of a walk whose collision efficiency is not established: the measured build uses the cycle rule of Section 3.4, which admits the cycles of Section 3.5. They are a ceiling on what a repaired table walk could reach with this arithmetic, not a rate of useful search.

The ratio of medians is

$$\frac{22.100934}{19.852741} = 1.11324346, \quad (33)$$

corresponding to an 11.3243% improvement. Each candidate run lasts approximately 22.8 seconds under the recorded counting convention. All five candidate rates exceed 22 billion updates/s. The time series is shown in Figure 2; the small downward trend is retained rather than hidden by selecting the fastest sample.

Post-run telemetry reports candidate SM clocks of 2,355–2,362 MHz, temperatures of 64–75 degrees Celsius, and instantaneous power readings of 544.64–584.69 W. The baseline samples report 2,400–2,415 MHz and 60–76 degrees Celsius. The candidate therefore ran about 1.6% slower in clock terms: its median reported SM clock is 2,362 MHz against 2,400 MHz for the baseline, with both variants drawing close to the 600 W limit. Dividing each sample’s rate by its own reported clock gives a ratio of medians of 1.134, a 13.4% gain per clock cycle compared with 11.32% in wall-clock terms. Equivalently, the candidate spends about 20.1 SM clock cycles per update and the baseline about 22.7. The wall-clock figure remains the headline because it is what a user of this power-limited device obtains. The normalized figure is indicative only: these snapshots are taken after each run and are not time-integrated traces. They do not support a joules-per-update or energy-efficiency result, and the frequencies were not fixed across the experiment. Runs with locked clocks, and runs

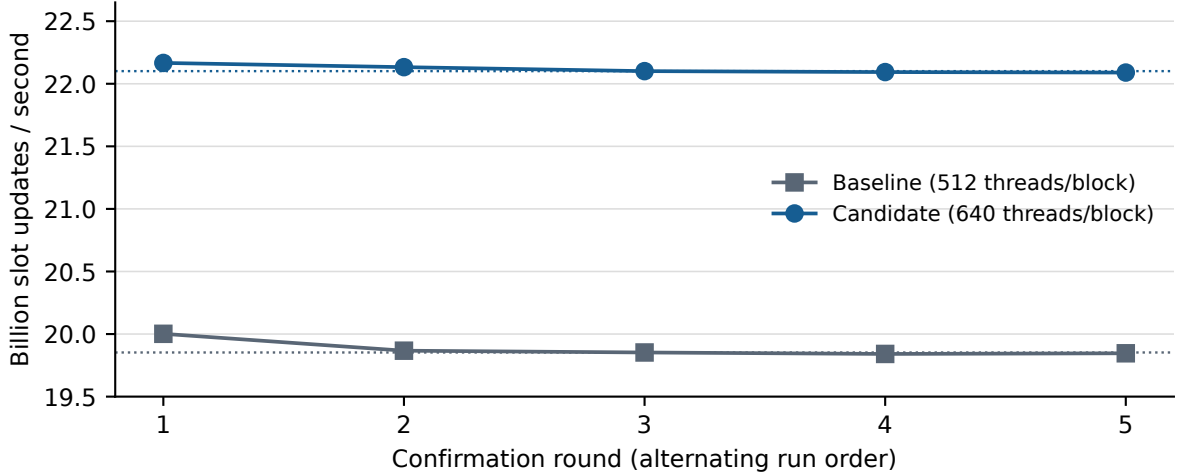


Figure 2: Accepted confirmation rates. Dotted lines show each variant’s median. The vertical axis is deliberately restricted to show within-session drift; numerical values and full counts are provided in Table 4.

long enough to reach thermal equilibrium, were not performed; each timed run lasts under 23 seconds while temperatures were still rising.

7.4 Distinguished-point collection

The separate DP34 candidate run completes the same 504,658,657,280 scheduled updates at 21.548499 billion/s and writes 12,387 records, totaling 396,384 bytes. It reports zero drops. Its rate is 97.5004% of the candidate benchmark median, a descriptive difference of approximately 2.50%. Because the collection result is one sample, this ratio is not a repeated estimate of collection overhead.

For correctness, the baseline collection run is forced to the same 120,320 workers and starting population. The two complete sorted report multisets have the same digest. This baseline geometry is deliberately different from its normal 96,256-worker benchmark geometry, so its 12.288201-billion-update/s collection rate is not used as a speedup denominator. Corpus equality requires equal initial populations; a fair throughput comparison requires appropriately specified configurations. These are different experimental requirements.

7.5 What the measured gain does and does not isolate

The baseline is the unmodified archived source tree, and the candidate is that tree with one patch. Table 5 lists every axis on which the two measured builds differ; all other build switches, including the native carryless multiplier, the weighted prefixes, and the batch size, are identical.

The confirmation compares only the two ends of this list, so it cannot attribute the gain. The multiplication-count reduction in Equation (20) is 4.41%, smaller than the measured 11.32%. The screening receipts give a partial attribution, because the candidate was built up in stages and every stage was timed against a baseline in the same session (Table 6).

Three observations follow. First, the inversion tree at the wider geometry is the largest step, about six percentage points. It depends on the geometry and on root placement: the same tree gave +3.67% at 512 threads, and an earlier unrotated four-warp tree on the unmodified 512-thread kernel was *slower* than its baseline by 3.06%. Second, restricted pointers help only together with the wider block; at 512 threads with interleaving enabled they measured

Axis	Baseline	Candidate
Inversion	One Itoh–Tsujii inverse per thread ($W = 1$)	Four-warp product tree with rotated roots ($W = 4$)
Polynomial square	Spread to 261 bits, then the general reducer	Compressed even/odd reduction, Section 4.4
Threads per block; warps per SM	512; 16	640; 20
Scalar slots on the device	1,540,096	1,925,120
Kernel buffer arguments	Parameter structure	Restricted pointers
Live metadata per slot	32-bit finished flag; 64-bit history	One byte; three 16-bit tags
Paired-product instruction interleaving	Enabled	Disabled
Registers per thread	116	94
Shared inverse scratch per block	None	12,800 bytes
Field bytes; persisting-L2 coverage	78,544,896; all	98,181,120; 83,886,080

Table 5: Every difference between the two measured builds, taken from the archived patch and from the configuration banner that each run prints. Forward selection pipelining is disabled in both.

Configuration (cumulative)	Arm, billion/s	Baseline, billion/s	Gain
Compact metadata, 640 threads, no interleaving	20.231	19.980	+1.26%
plus restricted pointers	20.564	20.095	+2.33%
plus rotated four-warp inversion	21.756	20.087	+8.31%
plus shorter spread before the general reducer	21.841	19.791	+10.36%
with compressed square reduction instead	22.611	20.071	+12.66%
Same configuration, five-sample confirmation	22.101	19.853	+11.32%

Table 6: Screening sequence. Each row is the median of three samples against the median of three baseline samples from the same session; the last row repeats Table 4. Sessions differ in clocks and temperature, so gains are comparable across rows only approximately.

−0.30%. Third, the compressed square adds three to four percentage points over the preceding configurations, and about two over the shorter spread alone. This step is larger than an operation count suggests. The root inverse converts to the normal basis, where squaring is a permutation, so the polynomial square runs only once per update, beside roughly five multiplications. The shorter routine probably also relieves register or scheduling pressure in the reverse pass; an isolated microbenchmark would be needed to confirm that.

This sequence is not a controlled factorial ablation. The stages were measured in different sessions, with three samples each, and no stage removes a single component from the final candidate. A controlled study would disable one row of Table 5 at a time within one alternating session, and would add an isolated microbenchmark of the two squaring routines. That study has not been run. The notebook also records unsuccessful alternatives, including binary-GCD inversion, larger per-thread batches, wider blocks, alternate shared-table layouts, and earlier collective schemes. An earlier primitive-probe include-path error was subsequently corrected; only the final receipt’s intended-header probes and confirmation samples are used for acceptance here.

Check	Recorded coverage	Outcome
Source integrity	119 archived files; measurement digest	All matched locally
Polynomial square identity	131 basis inputs, two edge cases, 10,000 random values	Passed locally
GPU field probes	1,133 squares, 1,133 inverses, 6,798 Frobenius maps	Receipt passed
GPU group inverse	1,024 inputs for each $W = 2, 4, 8, 16$; zeros and consecutive calls	Receipt passed
GPU metadata	1,025 lanes; byte flags; 200 history updates/lane	Receipt passed
Independent trail replay	300 reports with endpoint and length equality	Receipt passed
Complete replay corpus	32,688 records at matched population	Digests equal
DP34 collection corpus	12,387 records; 396,384 bytes	Digests equal
Checkpoint continuation	Baseline $\rightarrow$ candidate and reverse	Receipt passed
Compute Sanitizer	Device/runtime support unavailable	Not passed
Inherited <code>test-clmad</code>	Source/guard expectation failures remain	Not fully green

Table 7: Evidence classes. “Receipt passed” denotes a recorded GPU check, not a new run performed while preparing this manuscript. The algebraic identity check covers the full linear map; the other finite checks have their stated coverage.

8 Correctness evidence and practical limits

8.1 Evidence retained with the result

The independent field probes compare against `Ref<CfgF131>`, rather than merely comparing the candidate with a second use of the same optimized routine. The final receipt also includes the selected collective helper hash and probes the composed arithmetic implementation. The full replay comparison uses 65,536 starting scalar walks, six launches, and 16 steps per launch with reporting threshold 50. Equality is established for the complete sorted multiset of 32,688 persisted records, not just a sampled prefix.

Checkpoint tests split the same workload into three launches in one build and three in the other, then compare against uninterrupted execution. This tests observable continuation across the two memory representations. It does not establish compatibility with every future walk, campaign, or checkpoint format.

The final collection corpora themselves are summarized by retained hashes and counts in the receipt; the manuscript package does not claim to include the original binary corpus files. Reproducing those digests requires rerunning the documented client with the matching seeds and population. Similarly, the measured executable is identified by hash but is not claimed to have been reconstructed byte for byte by the local manuscript build.

8.2 Exceptional affine additions

The Frobenius walk has no exceptional additions on the subgroup. Its denominator is $d = x + \sigma^j(x) = x + x^{2^j}$, which vanishes exactly when $x \in \mathbb{F}_{2^j} \cap \mathbb{F}_{2^{131}} = \mathbb{F}_2$, because 131 is prime and $3 \leq j \leq 10$. A subgroup point has $\text{Tr}(x) = 0$ (Section 3.3), while $\text{Tr}(1) = 131 \bmod 2 = 1$, and $(0, 1)$ has order two. So neither $x = 0$ nor $x = 1$ occurs, $d \neq 0$ at every reachable point, and $\sigma^j(R) \neq \pm R$. The remainder of this subsection concerns the table walk.

The fast path implements Equation (17) without complete handling of $d = 0$, and the host table reference intentionally uses the corresponding raw addition. Agreement between those two paths therefore does not establish correct group behavior for an exceptional input. A zero denominator invalidates the affected thread’s local prefix chain: the batch product is

zero, the returned inverse is zero, and all 16 slots of that thread receive $\lambda = 0$. Cross-warp zero isolation protects other batches but does not resolve that case.

The exceptional set is small and explicit. The denominator vanishes exactly when the current point and the selected addend share an x coordinate, that is when $R = \pm\sigma^k(T_h)$ for the selected (h, k) . At most $2 \cdot 8 \cdot 131 = 2,096$ subgroup points have this form. Under the usual heuristic that visited points are uniform in the subgroup, the probability per step is at most

$$\frac{2,096}{\ell} \approx 3.1 \times 10^{-36}, \quad (34)$$

and the expected number of occurrences over an entire search of $2^{60.8}$ steps is about 6×10^{-18} . This bound is heuristic. It assumes that the public table seed does not correlate the table orbits with the starting points, and it is not a proven exclusion condition.

Detection needs no additional field arithmetic. The per-thread batch product is zero exactly when some $d_i = 0$ in that batch, and the collective helper already evaluates that zero test once per thread and step in order to isolate other warps. A complete policy is therefore to mark all 16 slots of a thread as finished whenever its returned inverse is zero, so that the ordinary reseed path replaces them. Discarding 16 trails with probability near 10^{-35} per step has no measurable effect on useful work. This policy is not implemented in the archived candidate and its cost was not timed; the archived timing and replay evidence establish the tested ordinary-domain behavior only.

8.3 Collision quality and the cost to solve

The familiar square-root collision estimate requires an appropriate model of the actual transition system. For the Frobenius walk that model has been checked: its iteration constant was measured on scaled curves (Section 3.2), it has no fruitless cycles, and at the campaign’s DP32 threshold, where a trail averages $2^{28.41}$ steps, a restart guard of 2^{32} steps discards about 0.01% of steps [12]. (The earlier guard of 2^{30} , only three mean trail lengths, discarded 15.6%; a scale model of the walk measured 14.2% against 14.2% predicted at the same cap-to-trail ratio.) The expected work is therefore $2^{60.91}$ – $2^{60.92}$ iterations. At 17.625650 billion iterations/s, Bailey et al.’s $2^{60.9} \approx 2.15 \times 10^{18}$ iterations take

$$\frac{2^{60.9}}{17.625650 \times 10^9 \text{ s}^{-1}} \approx 1.22 \times 10^8 \text{ s} \approx 3.9 \text{ GPU-years} \quad (35)$$

of expected work on one card, or about 34,000 GPU-hours. Rho’s running time is random, so a budget with 83% confidence of success is about 1.5 times the mean. This figure is a projection from a measured rate and a measured iteration constant, not a completed computation, and it excludes distinguished-point storage, collision checking and the final logarithm extraction, which this measurement does not cover.

For the table walk the same calculation does not hold. With the measured rule, Section 3.5 prices it at 4.85–6.26 times the Frobenius walk’s cost per solve at DP32, so 22.1 billion scheduled updates/s is worth less than 17.6 billion Frobenius iterations/s. With the repaired rule it is projected at 0.81–0.86 times, which would make it the cheaper walk, but that projection uses the old kernel’s paired rates and the repaired rule’s extra cold-path work is unmeasured. A paired GPU measurement of the repaired rule, with iterations-to-collision on the scaled $m = 41$ and $m = 83$ curves and the restart-guard loss at $m = 131$, would settle it. Until then we do not convert 22.100934 billion updates/s into a completion time.

8.4 Statistical and system limits

The experiment uses one GPU allocation and one toolchain/driver combination. Thermal drift is visible and the device is power-limited, so the two table-walk variants ran at different clocks,

and the Frobenius-walk receipt has no clock telemetry at all; collection has one repetition; the attribution in Table 6 rests on three-sample screens from separate sessions; no locked-clock run, energy integral, multi-device variance, or long-duration sustained-load distribution is available. Compute Sanitizer was unavailable on the device/runtime, and the inherited `test-clmad` suite has unresolved source-shape and architecture-guard expectations. These facts remain explicit even though the selected independent functional probes passed.

The repository also contains cloud persistence and worker-launch evidence. That evidence has a different scope and is excluded from the headline result. No fleet scaling factor, S3/RDS throughput guarantee, or completed challenge computation follows from this single-device acceptance receipt.

9 Reproducibility and artifact availability

The evidence boundary is two archived directories in the companion repository: the Frobenius-walk receipts in `ecc2k130/runner/research/production/` (confirmation, screening, source and build audit, and sanitizer attempts) and the table-walk candidate in `ecc2k130/runner/research/candidates/goal22/`. The accompanying manuscript package includes the table walk’s raw measurement receipt, acceptance record, source manifest, derived statistics, and a script that rechecks the archive, regenerates the square-reduction header and the performance figure, and tests the modulus for irreducibility.

The Frobenius-walk confirmation is driven by

```
modal run ecc2k130/runner/tune_frobenius.py --mode confirm \
--selected fused_chain_first
```

and each timed run is the campaign client with the receipt’s 26 build settings:

```
./ecc2k130 --packed --curve 131 --threads 120320 --steps 1024 \
--launches 128 --run-id 11000 --dp-weight 32 --verify 0
```

For the table walk, build the archive’s own target:

```
cd ecc2k130/runner/research/candidates/goal22
make gpu-goal22
./ecc2k130 --curve 131 --packed --bench \
--steps 1024 --launches 256 --verify 0
```

These are workload commands, not full protocols. Reproduction must restore the recorded compiler and GPU target, exclude warmups, alternate control and candidate over five rounds, and retain final counts; the receipts contain the complete build commands and binary identities.

Before public submission, an immutable source-and-evidence deposit will make these references independently accessible. The table walk’s open collision-quality question should accompany that deposit, so that a reader can reproduce both the strengths and the limits of the result.

10 Conclusion

On one RTX PRO 6000 Blackwell GPU, the ECC2K-130 iteration function of Bailey et al. runs at a median of 17.625650 billion iterations/s with distinguished-point collection on: about 280 times the 2010 GPU rate per card, and about 25 SM clock cycles per iteration against roughly 1,180. Because that walk’s iteration constant and cycle behavior are known, the rate converts directly into an expected cost of about 3.9 GPU-years for the whole challenge. A 130-bit binary-curve discrete logarithm that required a coordinated consortium in 2009 is now

within reach of a modest cloud budget. That says nothing new about deployed curves, which are far larger, but it is a concrete measure of how quickly a new instruction erodes a fixed security margin.

The arithmetic transfers beyond this curve. Polynomial state in the basis generated by the optimal-normal-basis element keeps `clmad` products and the walk’s normal-basis weight both cheap. Cross-warp product trees bring the inversion share to $5/(BW)$ multiplications per update. The even/odd square-and-reduce map, whose exactness we prove, accounted for three to four percentage points of the table walk’s measured gain. The step-by-step measurements show that `clmad` alone bought 22% and that most of the gain came from the layout work it made possible.

The faster table walk is not yet a faster solver. Its measured rule admits fruitless cycles that come from the Frobenius relation itself and trap about 12% of trails at DP34 and about half at the campaign’s DP32. A repaired rule exists and is projected at 0.81–0.86 times the Frobenius walk’s cost per solve. Measuring it on the card is the next step.

References

- [1] Daniel V. Bailey, Lejla Batina, Daniel J. Bernstein, Peter Birkner, Joppe W. Bos, Hsieh-Chung Chen, Chen-Mou Cheng, Gauthier van Damme, Giacomo de Meulenaer, Luis Julian Dominguez Perez, Junfeng Fan, Tim Güneysu, Frank Gürkaynak, Thorsten Kleinjung, Tanja Lange, Nele Mentens, Ruben Niederhagen, Christof Paar, Francesco Regazzoni, Peter Schwabe, Leif Uhsadel, Anthony Van Herrewege, and Bo-Yin Yang. Breaking ECC2K-130. Cryptology ePrint Archive, Report 2009/541, 2009. <https://eprint.iacr.org/2009/541>.
- [2] Daniel J. Bernstein, Hsieh-Chung Chen, Chen-Mou Cheng, Tanja Lange, Ruben Niederhagen, Peter Schwabe, and Bo-Yin Yang. ECC2K-130 on NVIDIA GPUs. Cryptology ePrint Archive, Report 2012/002, 2010. Progress in Cryptology – INDOCRYPT 2010, LNCS 6498, pp. 328–346. Revised version: <https://eprint.iacr.org/2012/002>.
- [3] Daniel J. Bernstein, Hsieh-Chung Chen, Chen-Mou Cheng, Tanja Lange, Ruben Niederhagen, Peter Schwabe, and Bo-Yin Yang. Usable assembly language for GPUs: A success story. Cryptology ePrint Archive, Report 2012/137, 2012. <https://eprint.iacr.org/2012/137>.
- [4] Daniel J. Bernstein, Susanne Engels, Tanja Lange, Ruben Niederhagen, Christof Paar, Peter Schwabe, and Ralf Zimmermann. Faster elliptic-curve discrete logarithms on FPGAs. Cryptology ePrint Archive, Report 2016/382, 2016. <https://eprint.iacr.org/2016/382>.
- [5] Daniel J. Bernstein and Tanja Lange. Type-II optimal polynomial bases. In *Arithmetic of Finite Fields – WAIFI 2010*, volume 6087 of *Lecture Notes in Computer Science*, pages 41–61. Springer, 2010. <https://eprint.iacr.org/2010/069>.
- [6] Daniel J. Bernstein, Tanja Lange, and Peter Schwabe. On the correct use of the negation map in the Pollard rho method. In *Public Key Cryptography – PKC 2011*, volume 6571 of *Lecture Notes in Computer Science*, pages 128–146. Springer, 2011. <https://cr.yp.to/elliptic/negation-20110102.pdf>.
- [7] Joppe W. Bos, Thorsten Kleinjung, and Arjen K. Lenstra. On the use of the negation map in the Pollard rho method. In *Algorithmic Number Theory – ANTS-IX*, volume 6197 of *Lecture Notes in Computer Science*, pages 66–82. Springer, 2010. https://doi.org/10.1007/978-3-642-14518-6_9.

- [8] Joppe W. Bos, Thorsten Kleinjung, Ruben Niederhagen, and Peter Schwabe. ECC2K-130 on Cell CPUs. In *Progress in Cryptology – AFRICACRYPT 2010*, volume 6055 of *Lecture Notes in Computer Science*, pages 225–242. Springer, 2010. <https://eprint.iacr.org/2010/077>.
- [9] Adam Buran. The 30 b/s objective, priced. Companion repository `aburan28/crypto, ecc2k130/THROUGHPUT-30B.md`, 2026. Instruction-visit counts and issue rates for the software, native-clmad and compact-state kernels.
- [10] Adam Buran. ECC2K-130 accepted Blackwell candidate: `warps4_poly_compact640`. Project research artifact, September 21, 2026. Local archive `ecc2k130/runner/research/candidates/goal22/`. Content hashes appear in Appendix A. Receipt, acceptance, and manifest are bundled with this manuscript; public archival deposit is pending.
- [11] Adam Buran. Frobenius pass fusion at 120,320 workers: Confirmation receipt. `ecc2k130/runner/research/production/2026-09-21-frobenius-fused-confirm.json`, 2026. With `ecc2k130/runner/research/FROBENIUS-FUSION.md`. Public archival deposit pending.
- [12] Adam Buran. Iterations per solve: the σ walk against the table walk. Companion repository `aburan28/crypto, ecc2k130/WALK-CONSTANT.md`, 2026. Iteration constants on scaled curves, fruitless-cycle enumeration and counts, trap-cost model, restart-guard measurements.
- [13] Adam Buran. Packed normal-basis CUDA backend. Companion repository `aburan28/crypto, ecc2k130/PAKED.md` and the per-step notes it links, 2026. Bitsliced control and packed-backend audit on RTX PRO 6000.
- [14] Adam Buran. Table-walk cycle escape v3. Companion repository `aburan28/crypto, ecc2k130/CYCLE-ESCAPE-V3.md`, 2026.
- [15] Junfeng Fan, Daniel V. Bailey, Lejla Batina, Tim Güneysu, Christof Paar, and Ingrid Verbauwhede. Breaking elliptic curve cryptosystems using reconfigurable hardware. In *International Conference on Field Programmable Logic and Applications (FPL 2010)*. IEEE, 2010. <https://doi.org/10.1109/FPL.2010.34>.
- [16] Robert P. Gallant, Robert J. Lambert, and Scott A. Vanstone. Improving the parallelized Pollard lambda search on anomalous binary curves. *Mathematics of Computation*, 69(232):1699–1705, 2000.
- [17] Toshiya Itoh and Shigeo Tsujii. A fast algorithm for computing multiplicative inverses in $GF(2^m)$ using normal bases. *Information and Computation*, 78(3):171–177, 1988. [https://doi.org/10.1016/0890-5401\(88\)90024-7](https://doi.org/10.1016/0890-5401(88)90024-7).
- [18] Peter L. Montgomery. Speeding the Pollard and elliptic curve methods of factorization. *Mathematics of Computation*, 48(177):243–264, 1987.
- [19] NVIDIA. Building faster cryptography with carryless multiplication in NVIDIA CUDA 13.3. NVIDIA Technical Blog, 2026. July 15, 2026. Benchmarks GHASH and $GF(2^{128})$ sum-check; no elliptic-curve results.
- [20] NVIDIA. Parallel Thread Execution ISA, version 9.4, 2026. Section 9.7.1.5, carryless multiply-add. Accessed September 21, 2026. <https://docs.nvidia.com/cuda/parallel-thread-execution/index.html#integer-arithmetic-instructions-clmad>.

- [21] John M. Pollard. Monte carlo methods for index computation (mod p). *Mathematics of Computation*, 32(143):918–924, 1978. <https://doi.org/10.1090/S0025-5718-1978-0491431-9>.
- [22] Paul C. van Oorschot and Michael J. Wiener. Parallel collision search with cryptanalytic applications. *Journal of Cryptology*, 12:1–28, 1999. Author manuscript available at <https://people.scs.carleton.ca/~paulv/papers/JoC97.pdf>.
- [23] Erich Wenger and Paul Wolfger. Solving the discrete logarithm of a 113-bit Koblitz curve with an FPGA cluster. In *Selected Areas in Cryptography – SAC 2014*, volume 8781 of *Lecture Notes in Computer Science*, pages 363–379. Springer, 2014. <https://eprint.iacr.org/2014/368>.
- [24] Erich Wenger and Paul Wolfger. Harder, better, faster, stronger: Elliptic curve discrete logarithm computations on FPGAs. *Journal of Cryptographic Engineering*, 6, 2016. <https://eprint.iacr.org/2015/143>.
- [25] Michael J. Wiener and Robert J. Zuccherato. Faster attacks on elliptic curve cryptosystems. In *Selected Areas in Cryptography – SAC '98*, volume 1556 of *Lecture Notes in Computer Science*, pages 190–200. Springer, 1999.

A Artifact identities and implementation map

Object	SHA-256
Measurement receipt	840b6071d202393a958b00317be58d90f b2a9367363659d18daaaebba2dc4d82
Candidate executable	da24894bdef26c638b243790a3bf4cf1 66feecac27221cd8c035191dc20d9e9a
Baseline executable	b62f1c6162164dcea69c7478525f8a0a f65bb295420ce919da9830f4fc1a0500
Replay corpus (sorted)	db31c5c36206192fe8fda4cba58152e4 6620d2ffe5e4d22605e7f7d34909162e
DP34 corpus (sorted)	334b830fc6d3bf5801013203010bb030 baddfdc5950f11c8a881a5c95ff9c092

The recorded GPU UUID is GPU-080ac0e6-6dbe-3064-5ce3-7fe34abd5193. The archived candidate name, hashes, and source manifest identify this study even if the live build profile changes.

Paper component	Archived implementation
Curve and field constants	<code>generated/eccF131.h</code>
Reference table and raw update	<code>include/ref.h</code> , <code>TableWalk<Cfg></code>
Phase, pivot, cycle tags	<code>include/tablewalk.h</code>
Device selection and addends	<code>include/packedtablewalk.cuh</code>
Algorithm 1 forward/reverse passes	<code>include/packedkernels.cuh</code> , <code>walk</code>
Four-warp inverse	<code>include/collective_inverse.cuh</code> , <code>goal22CollectiveInverse<4></code>
Compressed square reduction	<code>research/gen_square_reduce.py</code> ; <code>include/square_reduce.h</code>
Packed state and metadata	<code>include/packedcompactstate.cuh</code> ; <code>include/compact_metadata.cuh</code>
Timer and update accounting	<code>src/main.cu</code> , completed-run output

Table 8: Mapping between manuscript algorithms and the frozen source. All paths in this table are relative to the archived candidate directory.

B Zero handling and product-tree invariant

For nonzero leaves $d_0, \dots, d_{W-1}$, each upward tree node stores the product of its descendant leaves. If a node with children L, R holds $(LR)^{-1}$, its child inverses are $R(LR)^{-1} = L^{-1}$ and $L(LR)^{-1} = R^{-1}$. The downward pass therefore returns each original leaf inverse. Replacing a zero leaf by one before this construction keeps the product invertible; restoring zero at that output implements the helper’s explicit convention $\text{inv}(0) = 0$.

The logical warp permutation is bijective within each barrier group and leaves lane positions unchanged. Each group consequently runs 32 separate product trees, and no operand crosses a group boundary. This algebraic invariant explains the returned values. Race freedom and barrier participation still rely on the actual synchronization implementation and are supported by finite device tests, with the sanitizer limitation reported in Table 7.